

Homework 2

Interconnect Topologies and Parallelism

Introduction

This write-up solves the two questions of Homework 2 step by step. Throughout, the workload is a fully-connected layer

$$Y = XW, \quad X \in \mathbb{R}^{B \times N}, \quad W \in \mathbb{R}^{N \times M}, \quad Y \in \mathbb{R}^{B \times M},$$

with $N = M = 2^{17}$, batch size $B = 1024 = 2^{10}$, and FP32 values (4 B each). The backward pass needs

$$dW = X^\top dY \in \mathbb{R}^{N \times M}, \quad dX = dY W^\top \in \mathbb{R}^{B \times N}.$$

Recurring tensor sizes.

- $|X| = |dX| = |Y| = |dY| = B \cdot N \cdot 4 = 2^{29} \text{ B} = 0.5 \text{ GiB}$.
- $|W| = |dW| = N \cdot M \cdot 4 = 2^{36} \text{ B} = 64 \text{ GiB}$.

Collective cost formulas (from Week 2–3 lecture, slide 45). For a tensor of total size D distributed across P GPUs, with per-direction bandwidth BW on the communicating links:

$$T_{\text{all-reduce}} = 2 \frac{P-1}{P} \cdot \frac{D}{\text{BW}} \quad (\text{reduce-scatter} + \text{all-gather})$$

$$T_{\text{all-gather}} = \frac{P-1}{P} \cdot \frac{D}{\text{BW}}.$$

These hold for both the decentralized (fully-connected) version under a per-GPU bandwidth budget and the ring version on a ring of P nodes.

Question 1. Crossbar Interconnect: Data & Tensor Parallelism (8 pts)

Setup. 8 GPUs on a crossbar. Each GPU has a per-GPU inbound bandwidth of 64 GiB/s and an outbound bandwidth of 64 GiB/s (full-duplex). Note this is a *per-GPU* budget, shared across all simultaneous transfers – different from HW1 Q1, where each pair of GPUs had its own dedicated link. Hence

$$\text{BW} = 64 \text{ GiB/s} = 2^{36} \text{ B/s}.$$

Q1a. Data-parallel implementation (3 pts)

Assumptions.

- GPU i holds full W (replicated) and shard $X^{(i)} \in \mathbb{R}^{B/P \times N}$, $B/P = 128$ samples.
- $Y^{(i)} = X^{(i)}W$ is produced and consumed locally; $dX^{(i)} = dY^{(i)}W^\top$ uses $dY^{(i)}$ on the same GPU.

- All-reduce is decentralized; per-GPU bandwidth is shared across all simultaneous transfers.

Step 1: Forward derivation. GPU i computes $Y^{(i)} = X^{(i)}W$ locally. $Y^{(i)}$ stays on GPU i , so there is no forward communication.

$$T_F = 0 \text{ ms.}$$

Step 2: Backward derivation. On GPU i :

$$dX^{(i)} = dY^{(i)}W^\top \quad (\text{local, } W \text{ is replicated}),$$

$$dW_{\text{partial}}^{(i)} = (X^{(i)})^\top dY^{(i)} \in \mathbb{R}^{N \times M}.$$

The true weight gradient $dW = \sum_{i=1}^P dW_{\text{partial}}^{(i)}$ requires aggregating and redistributing the P partials – an **all-reduce of dW** .

Step 3: Communication cost.

$$D = |dW| = 2^{36} \text{ B} = 64 \text{ GiB}, \quad P = 8, \quad \text{BW} = 2^{36} \text{ B/s.}$$

$$T_B = 2 \cdot \frac{P-1}{P} \cdot \frac{D}{\text{BW}} = 2 \cdot \frac{7}{8} \cdot \frac{2^{36}}{2^{36}} = \frac{7}{4} \text{ s} = \mathbf{1750 \text{ ms.}}$$

Total. $T_F + T_B = 0 + 1750 = \boxed{1750 \text{ ms}}.$

Q1b. Tensor-parallel implementation (5 pts)

Assumptions. We use the Megatron-LM *row-parallel* linear layer (Week 4 Megatron-LM slides) – the variant requiring *all-reduce* in the forward pass and *all-gather* in the backward pass:

- Weights split row-wise: $W = \begin{bmatrix} W_1 \\ \vdots \\ W_P \end{bmatrix}$, $W_i \in \mathbb{R}^{N/P \times M}$, with $N/P = 2^{17}/8 = 2^{14}$.
- Inputs split column-wise (along the input-feature dim): $X = [X_1, \dots, X_P]$, $X_i \in \mathbb{R}^{B \times N/P}$.
- Each GPU i holds X_i and W_i .
- Decentralized all-reduce / all-gather on the crossbar with per-GPU bandwidth $\text{BW} = 64 \text{ GiB/s}$.

Step 1: Forward. Because $XW = \sum_{i=1}^P X_i W_i$, GPU i computes the partial output

$$Y_{\text{partial}}^{(i)} = X_i W_i \in \mathbb{R}^{B \times M} \quad (\text{local}).$$

An **all-reduce of Y** accumulates the partials on every GPU:

$$D = |Y| = 2^{29} \text{ B} = 0.5 \text{ GiB.}$$

$$T_F = 2 \cdot \frac{P-1}{P} \cdot \frac{D}{\text{BW}} = 2 \cdot \frac{7}{8} \cdot \frac{2^{29}}{2^{36}} = \frac{7}{512} \text{ s} \approx \mathbf{13.67 \text{ ms.}}$$

Step 2: Backward. The all-reduce leaves $dY \in \mathbb{R}^{B \times M}$ replicated on every GPU.

Weight gradient. GPU i owns rows i of W , so

$$dW_i = X_i^\top dY \in \mathbb{R}^{N/P \times M}$$

is computed locally from X_i and dY . *No communication.*

Input gradient. From $dX = dY W^\top$ and the row-wise split, each GPU computes its slice locally: $dX_i = dY W_i^\top \in \mathbb{R}^{B \times N/P}$. An **all-gather of the dX_i slices** recovers the full dX :

$$D = |dX| = 2^{29} \text{ B} = 0.5 \text{ GiB}.$$

$$T_B = \frac{P-1}{P} \cdot \frac{D}{\text{BW}} = \frac{7}{8} \cdot \frac{2^{29}}{2^{36}} = \frac{7}{1024} \text{ s} \approx \mathbf{6.84 \text{ ms}}.$$

Step 3: Total.

$$T_F + T_B = \frac{7}{512} + \frac{7}{1024} = \frac{21}{1024} \text{ s} \approx \boxed{20.51 \text{ ms}}.$$

Question 2. 2D Torus Interconnect: Combined Data & Model Parallelism (12 pts)

Setup. A 4×4 2D torus with *unidirectional* links of 16 GiB/s each. The 16 GPUs are organised as 4 rows $\times$ 4 columns. Each row is a data-parallel group of size $P_r = 4$ (handles 256 of the 1024 samples), and each column is a tensor-parallel group of size $P_c = 4$ (handles $M/4 = 2^{15}$ output activations).

$$\text{BW} = 16 \text{ GiB/s} = 2^{34} \text{ B/s}.$$

Assumptions (shared).

- GPU (r, c) holds the weight slice $W_{(c)} \in \mathbb{R}^{N \times M/4}$ (column-parallel weights, replicated down each column).
- Weight slice size: $|W_{(c)}| = N \cdot (M/4) \cdot 4 = 2^{17} \cdot 2^{15} \cdot 4 = 2^{34} \text{ B} = 16 \text{ GiB}$.
- Row-direction links (horizontal) and column-direction links (vertical) are physically disjoint $\Rightarrow$ a ring along a row can run *concurrently* with a ring along a column.
- Ring all-reduce on $P = 4$: $T = 2 \cdot \frac{3}{4} \cdot \frac{D}{\text{BW}} = \frac{3D}{2\text{BW}}$. Ring all-gather on $P = 4$: $T = \frac{3}{4} \cdot \frac{D}{\text{BW}}$.

Two readings of the initial input layout

The assignment fixes per-row samples and per-column outputs but does not say how the 256 samples in a row are distributed across its four GPUs. Two natural choices:

Reading A (primary, used for the headline number). X is sharded across the full 4×4 grid along both axes:

$$X^{(r,c)} \in \mathbb{R}^{256 \times N/4} \quad (\text{batch by row, features by column}).$$

Each GPU initially stores 1/16 of X ; a ring all-gather within each row reconstructs $X^{(r)}$ before the local GEMM.

Reading B (alternative). Within each row r , the 256 samples are already *replicated* across all four column GPUs, so every GPU (r, c) holds the full $X^{(r)} \in \mathbb{R}^{256 \times N}$ and the forward pass needs no communication.

Forward pass (4 pts)

Reading A. A ring all-gather is performed within each of the 4 rows, in parallel (the 4 row-rings use disjoint horizontal links). After the all-gather, every GPU (r, c) has the full row-slice $X^{(r)} \in \mathbb{R}^{256 \times N}$ and computes locally

$$Y^{(r,c)} = X^{(r)} W_{(c)} \in \mathbb{R}^{256 \times M/4},$$

the $256 \text{ samples} \times 2^{15}$ output activations that this GPU owns. The communicated tensor per row is the full row-slice of X :

$$D = 256 \cdot N \cdot 4 = 2^8 \cdot 2^{17} \cdot 2^2 = 2^{27} \text{ B} = 128 \text{ MiB}.$$

$$T_F^A = \frac{P-1}{P} \cdot \frac{D}{\text{BW}} = \frac{3}{4} \cdot \frac{2^{27}}{2^{34}} = \frac{3}{512} \text{ s} \approx \mathbf{5.86 \text{ ms}}.$$

Reading B. $X^{(r)}$ is already on each GPU of row r , so each GPU directly computes $Y^{(r,c)} = X^{(r)} W_{(c)}$ with no communication.

$$T_F^B = 0 \text{ ms}.$$

Backward pass (8 pts)

Setup. GPU (r, c) receives $dY^{(r,c)} \in \mathbb{R}^{256 \times M/4}$ (the gradient corresponding to its own output tile). Two gradients have to be produced: dW and dX .

Step 1: Weight gradient dW . Since the weight is sharded by output dim, GPU (r, c) is responsible for the slice $dW_{(c)} \in \mathbb{R}^{N \times M/4}$, where

$$dW_{(c)} = X^\top dY_{(c)} = \sum_{r=0}^3 (X^{(r)})^\top dY^{(r,c)}.$$

Each GPU (r, c) computes its partial

$$dW_{(c),\text{partial}}^{(r)} = (X^{(r)})^\top dY^{(r,c)} \in \mathbb{R}^{N \times M/4}.$$

The r -sum (over 4 rows) is performed by a **ring all-reduce within each column**; there are 4 independent column rings, all using disjoint vertical links.

$$D = |W_{(c)}| = 2^{34} \text{ B} = 16 \text{ GiB}.$$

$$T_{dW} = 2 \cdot \frac{P-1}{P} \cdot \frac{D}{\text{BW}} = \frac{3}{2} \cdot \frac{2^{34}}{2^{34}} = \frac{3}{2} \text{ s} = \mathbf{1500 \text{ ms}}.$$

Step 2: Input gradient dX . Within a row, summing over output columns gives the row's input gradient:

$$dX^{(r)} = \sum_{c=0}^3 dY^{(r,c)} W_{(c)}^\top.$$

Each GPU (r, c) computes its partial

$$dX_{\text{partial}}^{(r,c)} = dY^{(r,c)} W_{(c)}^\top \in \mathbb{R}^{256 \times N}.$$

The c -sum is performed by a **ring all-reduce within each row** (4 independent row rings, disjoint horizontal links).

$$D = 256 \cdot N \cdot 4 = 2^{27} \text{ B} = 128 \text{ MiB}.$$

$$T_{dX} = 2 \cdot \frac{P-1}{P} \cdot \frac{D}{\text{BW}} = \frac{3}{2} \cdot \frac{2^{27}}{2^{34}} = \frac{3}{256} \text{ s} \approx \mathbf{11.72 \text{ ms}}.$$

Step 3: Concurrency. The dW all-reduce uses only *column* (vertical) links and the dX all-reduce uses only *row* (horizontal) links. They share no link, so they overlap entirely:

$$T_B = \max(T_{dW}, T_{dX}) = \max(1500, 11.72) = \mathbf{1500 \text{ ms}}.$$

Total for Question 2

Reading A (primary).

$$T = T_F^A + T_B = \frac{3}{512} + \frac{3}{2} \approx 5.86 + 1500 \approx \boxed{1505.86 \text{ ms}}.$$

Reading B (alternative).

$$T = 0 + 1500 = \boxed{1500 \text{ ms}}.$$

In both readings, the dW all-reduce over the 16 GiB weight slice sets the total.

Summary

Question	Forward	Backward	Total
Q1a – crossbar, data-parallel	0 ms	1750 ms	1750 ms
Q1b – crossbar, tensor-parallel	$\approx 13.67 \text{ ms}$	$\approx 6.84 \text{ ms}$	$\approx \mathbf{20.51 \text{ ms}}$
Q2 (Reading A) – 4×4 torus, DP×TP	$\approx 5.86 \text{ ms}$	1500 ms (concurrent)	$\approx \mathbf{1506 \text{ ms}}$
Q2 (Reading B) – 4×4 torus, DP×TP	0 ms	1500 ms (concurrent)	1500 ms